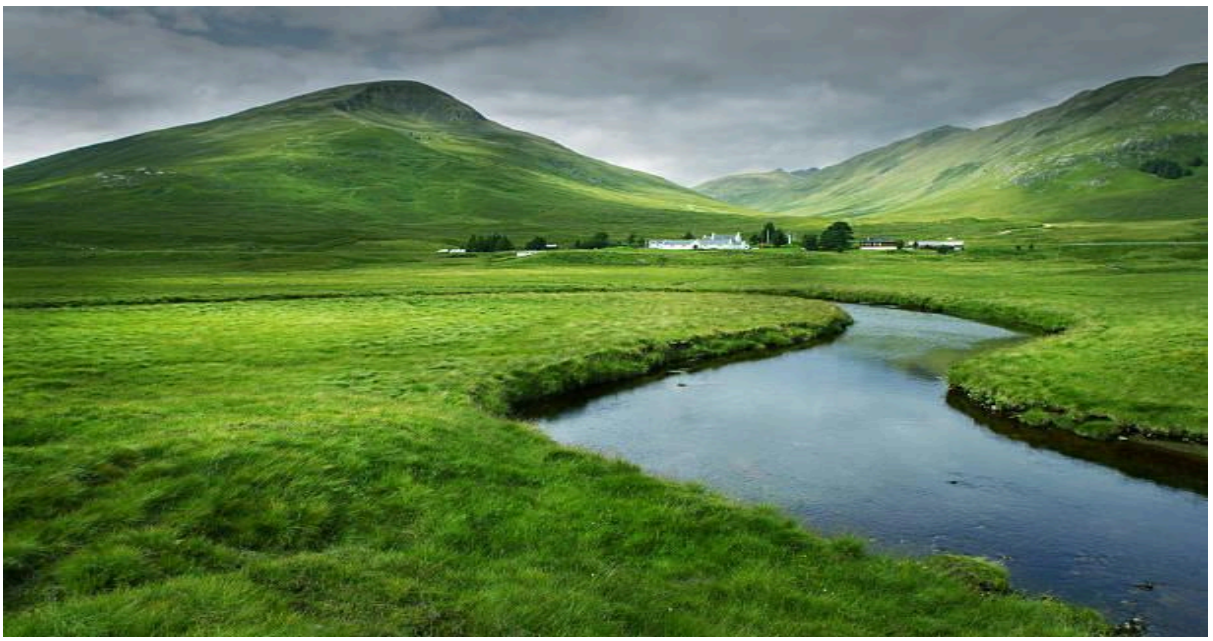

Data Analytics Project: Scottish Hills Dataset

Dataset Used: scottish_hills.csv

This document outlines a structured, end-to-end data analysis project based on the Scottish Hills dataset. The project is divided into five scenarios and it contains important fields such as hill names, their heights, Latitude, Longitude, and the Osgrid. This dataset serves as the foundation for performing data cleaning, analysis, and visualization tasks.

Project Goal: The goal of this project is to analyze and visualize the Scottish Hills dataset to uncover patterns in hill heights and regional distribution.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning (🟢 Easy)

Description:

This scenario focuses on importing the Scottish Hills dataset using the **Pandas** library and understanding its basic structure. The first few rows and column names are displayed to get an overview of the data. It then checks for missing values in important columns like Height and Region. Missing values are handled by filling Height with its mean and Region with its most frequent value (mode). Finally, the Height column is converted into a numeric format to ensure accurate analysis in later steps.

Task	Description
1.	Load the `scottish_hills.csv` dataset using Pandas.
2.	Display: ○ First 5 rows ○ Column names
3.	Check for missing values in: ○ Height ○ Region
4.	Fill missing values: ○ Height → use mean ○ Region → use mode
5.	Convert Height column to numeric if required..

```
PS C:\Users\Windows\Desktop\Python Practice Task\Analytics Tasks\Project 3> & C:\Users\Windows\Ap
pData\Local\Python\bin\python.exe "c:/Users/Windows/Desktop/Python Practice Task/Analytics Tasks/
Project 3/main.py"

 First 5 rows :
                    Hill Name  Height   Latitude  Longitude    Osgrid      Region
0       A' Bhuidheanach Bheag   936.0  56.870342  -4.199001  NN660775  South-East
1              A' Chailleach   997.0  57.693800  -5.128715  NH136714  North-West
2              A' Chailleach   929.2  57.109564  -4.179285  NH681041  North-East
3  A' Chraileag (A' Chralaig)  1120.0  57.184186  -5.154837  NH094147  North-West
4            A' Ghlas-bheinn   918.0  57.255090  -5.303687  NH008231  North-West

 Column Names:
 Index(['Hill Name', 'Height', 'Latitude', 'Longitude', 'Osgrid', 'Region'], dtype='str')
```

Fig 1.1 : First 5 rows() and Column names

Conclusion:

This ensures that the dataset is properly prepared before performing any analysis. It identifies and handles missing values in important columns, improving data completeness. Filling values using mean and mode maintains consistency without losing information. Converting the Height column into a numeric format helps avoid errors during calculations. These steps enhance the overall quality and reliability of the dataset. As a result, the data becomes suitable for accurate visualization and deeper insights.

Scenario 2: Line Graph + Save (🟢 Easy)

Description:

This scenario focuses on visualizing hill heights using a simple line graph for better understanding of variations. The dataset is filtered to select Hill Name and Height columns, and only the first 10 records are considered for clarity. The Height values are converted into a NumPy array to enable efficient plotting. A line graph is created with index values on the X-axis and heights on the Y-axis to show trends. Proper titles and axis labels are added to improve readability. Finally, the graph is saved as an image file for future reference and reporting.

Task	Description
1.	Select: ○ Hill Name ○ Height
2.	Take First 10 rows only.
3.	Convert Height into a NumPy array
4.	Plot a line graph: ○ X-axis → index (0–9) ○ Y-axis → Height
5.	Add title and labels.

Graph :

The line graph displays the heights of the first 10 hills using index values on the X-axis and height on the Y-axis. It shows how the heights increase or decrease across the selected hills, making trends easy to observe.

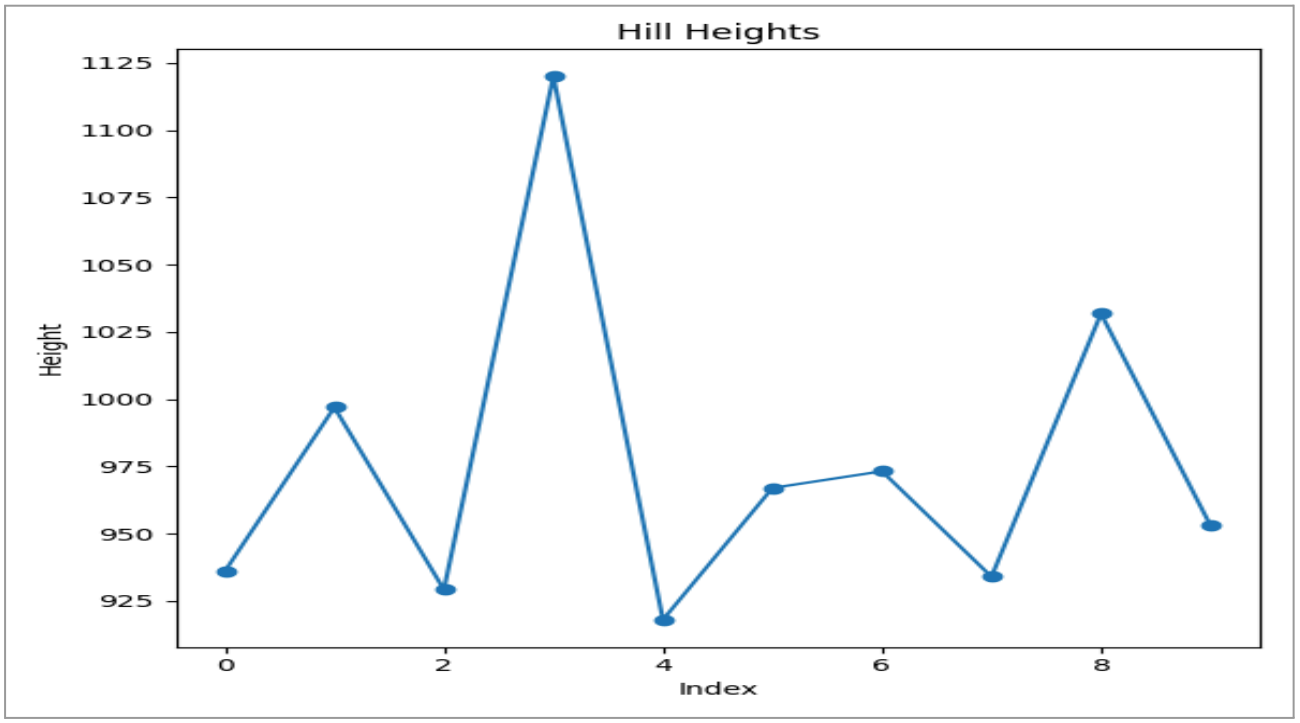

Fig 2.1 : Line Graph for Hill Heights

Conclusion:

This demonstrates how to visualize hill height data effectively using a line graph. It helps in identifying trends and variations across the selected hills. Converting the data into a NumPy array ensures efficient processing and smooth plotting. Adding titles and labels improves the clarity and readability of the graph. Saving the graph as an image makes it useful for reports and future reference.

Scenario 3: : Filtering + Bar Chart + Save (🟡 Medium)

Description:

This scenario focuses on analyzing tall hills by applying filtering techniques to the dataset. Hills with a height greater than 900 are selected to narrow down the analysis. The data is then grouped by Region to count how many tall hills are present in each area. These counts are converted into NumPy arrays for efficient handling. A bar chart is plotted with regions on the X-axis and the number of tall hills on the Y-axis. Labels are rotated for better readability, and the graph is saved for reporting purposes.

Task	Description
1.	Filter hills where: ○ Height > 900
2.	Count number of hills per Region.
3.	Select top regions
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: ○ X-axis → Region ○ Y-axis → count
6.	Rotate labels for clarity.

Graph :

The bar chart represents the number of tall hills (height > 900) across different regions. The X-axis displays the regions, while the Y-axis shows the count of hills in each region. Taller bars indicate regions with a higher number of tall hills. This visualization makes it easy to compare and identify regions with the most high-elevation hills.

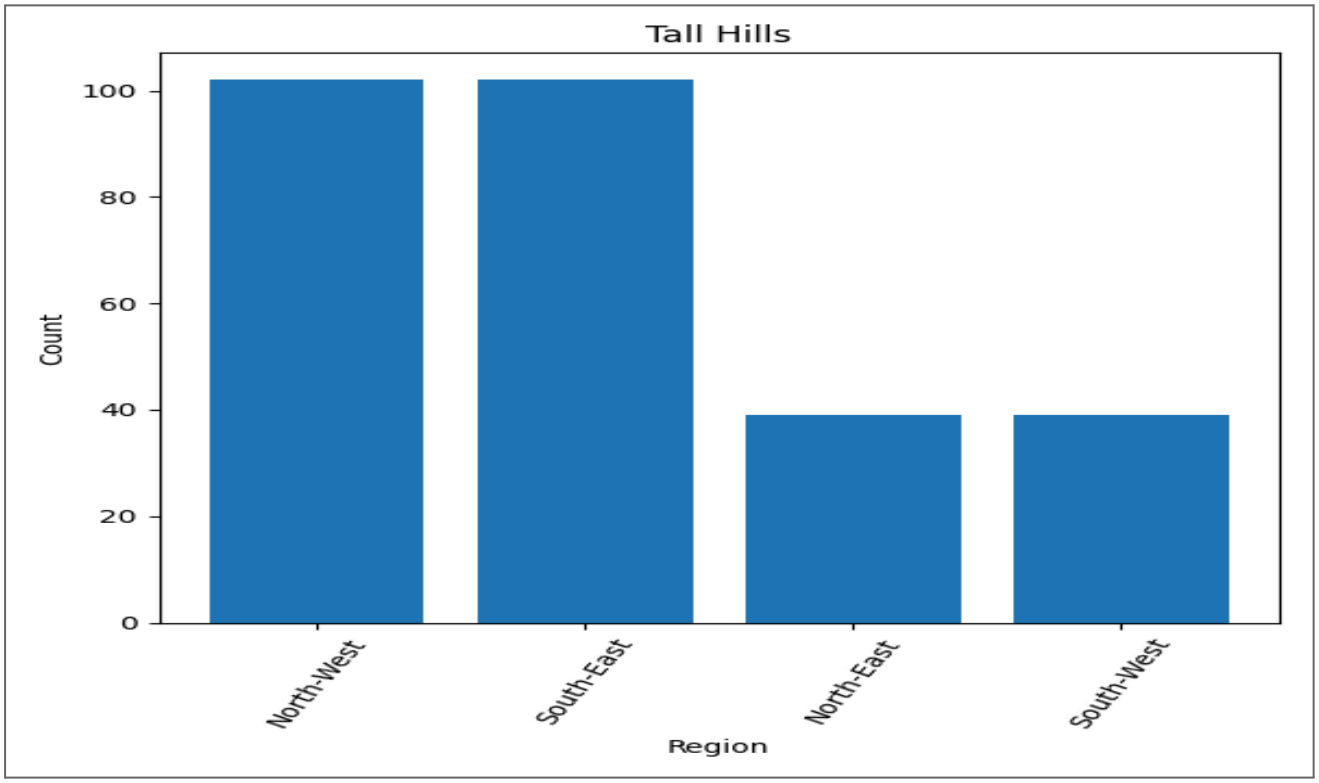

Fig 3.1 : Bar Chart for Tall Hills

Conclusion:

This highlights how filtering data can help focus on specific insights, such as identifying tall hills. Grouping the filtered data by region makes it easier to compare distributions across different areas. The bar chart provides a clear visual comparison of the number of tall hills in each region. Converting the data into NumPy arrays ensures efficient processing and plotting. Saving the graph allows it to be reused in reports and presentations. Overall, it enhances data analysis through targeted filtering and visualization.

Scenario 4: Pie Chart (Region Distribution) + Save (🟡 Medium)

Description:

This scenario focuses on understanding how hills are distributed across different regions using a pie chart. The dataset is grouped by Region to calculate the number of hills in each area. To maintain clarity, only the top 5 regions with the highest counts are selected. Labels and percentage values are added to clearly show each region's share. The pie chart provides an easy-to-understand visual comparison of distribution. Finally, the graph is saved as an image for use in reports and presentations.

Task	Description
1.	Count the number of hills per Region.
2.	Select top 5 regions
3.	Prepare labels and values.
4.	Plot a pie chart.
5.	Add percentage labels.

Graph :

The pie chart illustrates the distribution of hills among the top 5 regions. Each slice represents a region, with its size indicating the number of hills in that area. Larger slices show regions with a higher concentration of hills. This visualization makes it easy to compare regional contributions at a glance.

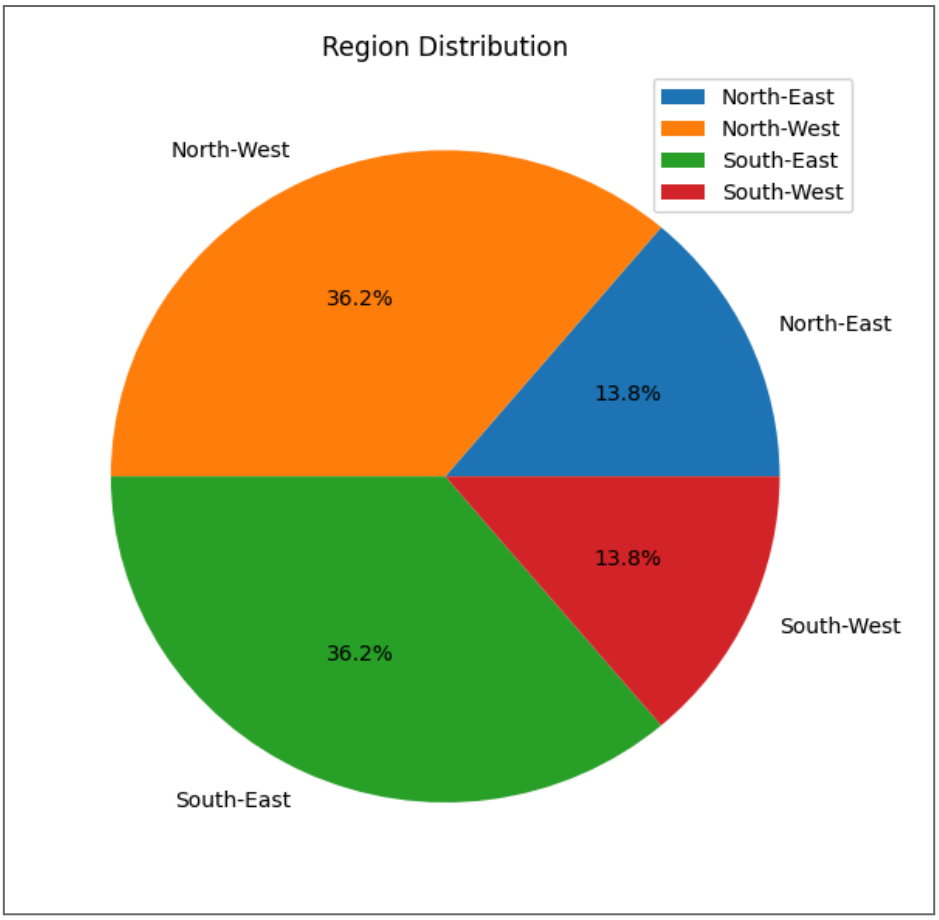

Fig 4.1: Pie Chart For Region Distribution

Conclusion:

This effectively visualizes how hills are distributed across different regions using a pie chart. It helps in understanding the proportion of hills in each area at a glance. Selecting the top regions keeps the chart clear and easy to interpret. Adding percentage labels improves readability and comparison. Saving the chart makes it useful for reports and presentations. Overall, it provides a simple and effective way to analyze regional distribution.

Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

Description:

This scenario focuses on performing deeper analysis by combining feature creation, NumPy operations, and multiple visualizations. A new column called Height Category is created to classify hills into Very High, High, and Moderate based on their heights. The Height column is converted into a NumPy array to calculate differences between consecutive values using np.diff(). A line graph is plotted to observe overall height trends across all hills. A stacked bar chart is used to compare height categories across different regions. Finally, a histogram is created to understand the distribution of hill heights, providing comprehensive insights into the dataset.

Task	Description
1.	Feature Engineering 1. Create a new column: ○ score_category: ■ score >= 9 → "Excellent" ■ 7 <= score < 9 → "Good" ■ < 7 → "Average" 2. Convert editors_choice: ○ Y → 1, N → 0
2.	NumPy Analysis 3. Use NumPy to: ○ Calculate yearly score growth using np.diff() on average yearly scores
3.	Visualizations Line Graph 4. Plot trend of: ○ Average score per release_year ►► Stacked Bar Chart 5. Show count of: ○ score_category per release_year ►► Histogram 6. Plot distribution of: ○ score
4.	Save All Graphs plt.savefig("score_trend.png") plt.savefig("score_category_stacked.png") plt.savefig("score_distribution.png"
5.	Insights: Identify: ● Which years had highest scores ● Whether high scores increased over time ● If editors_choice correlates with high scores

Graph :

1. **Line Graph :**

The line graph shows the overall trend of hill heights across the dataset. The X-axis represents the index of hills, while the Y-axis shows their heights. It helps identify patterns, fluctuations, and variations in elevation.

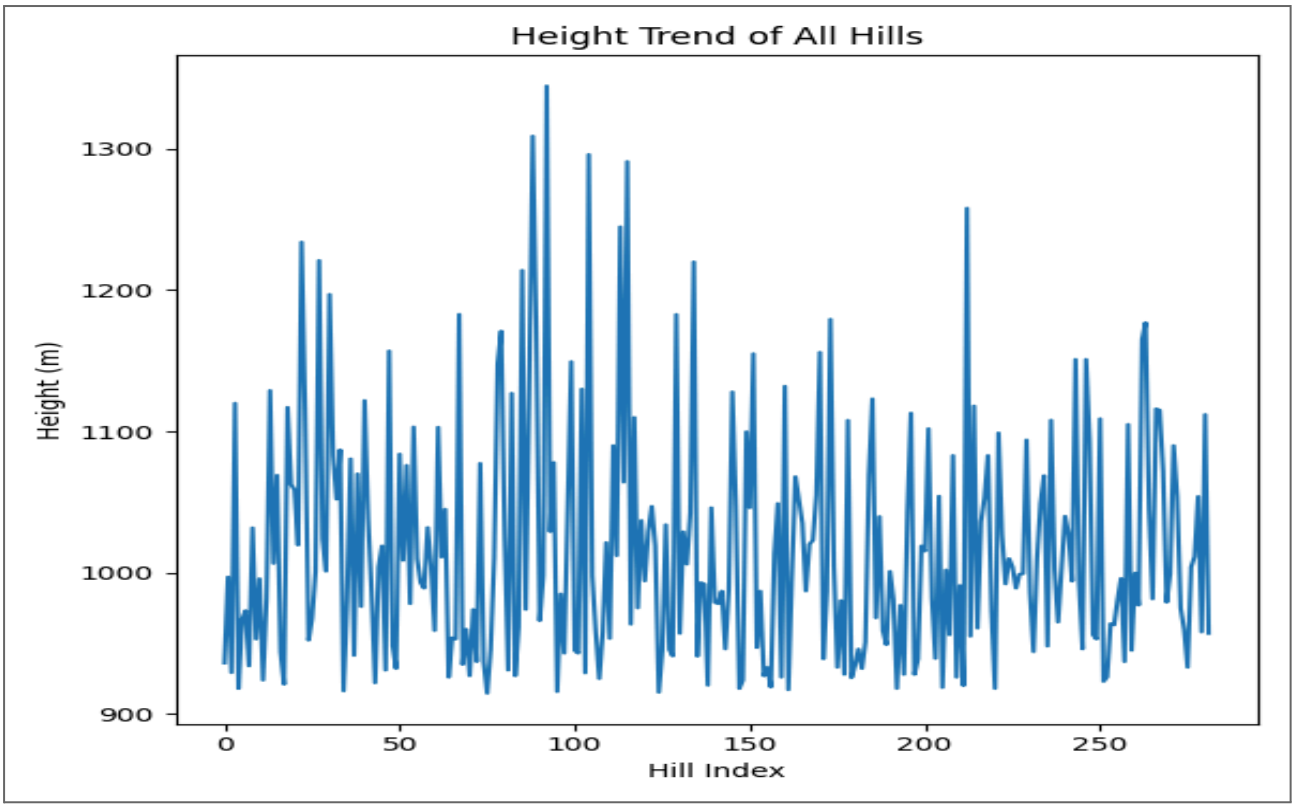

Fig 5.1 : Line Graph for Height Trend of All Hills

2. **Stacked Bar Chart :**

The stacked bar chart shows the count of different height categories (Very High, High, Moderate) for each region. Each bar is divided into segments representing these categories. It helps compare how height groups are distributed across regions.

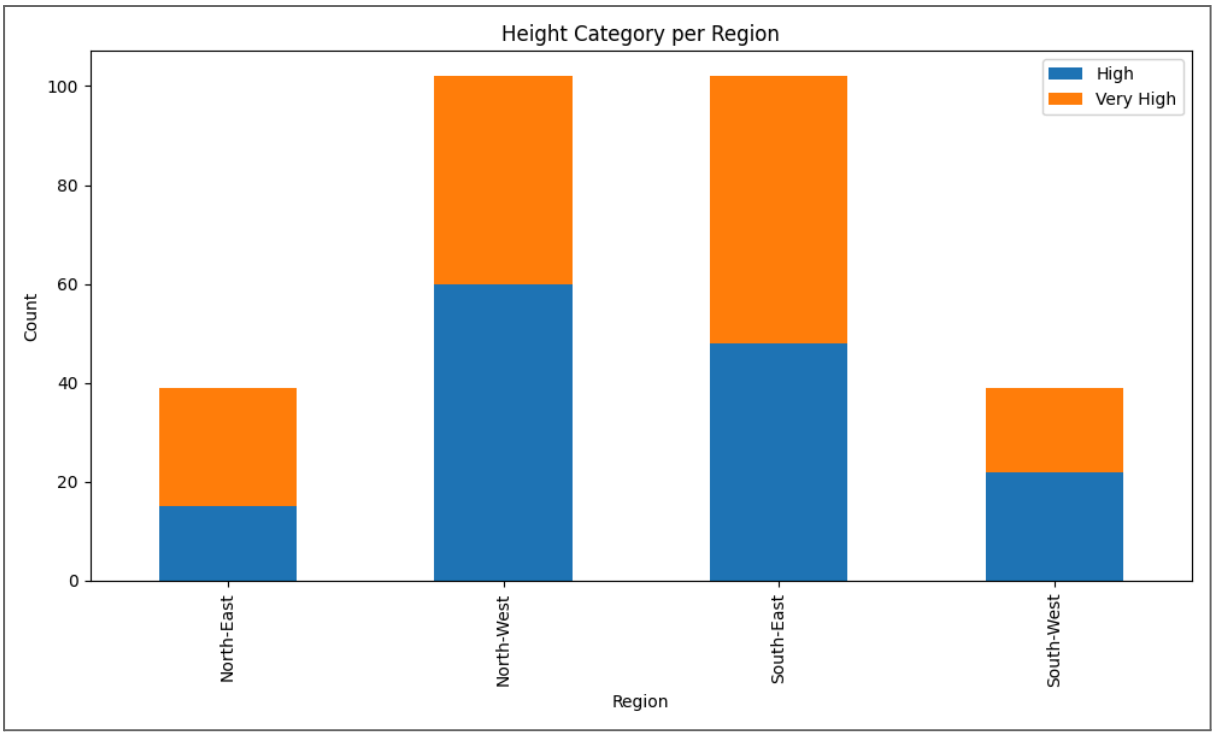

Fig 5.2: Stacked Bar Chart for Height Category per Region

3. **Histogram :**

The histogram shows the distribution of hill heights by grouping them into ranges (bins). The X-axis represents height ranges, while the Y-axis shows the frequency of hills in each range. It helps identify common height intervals and overall spread of the data.

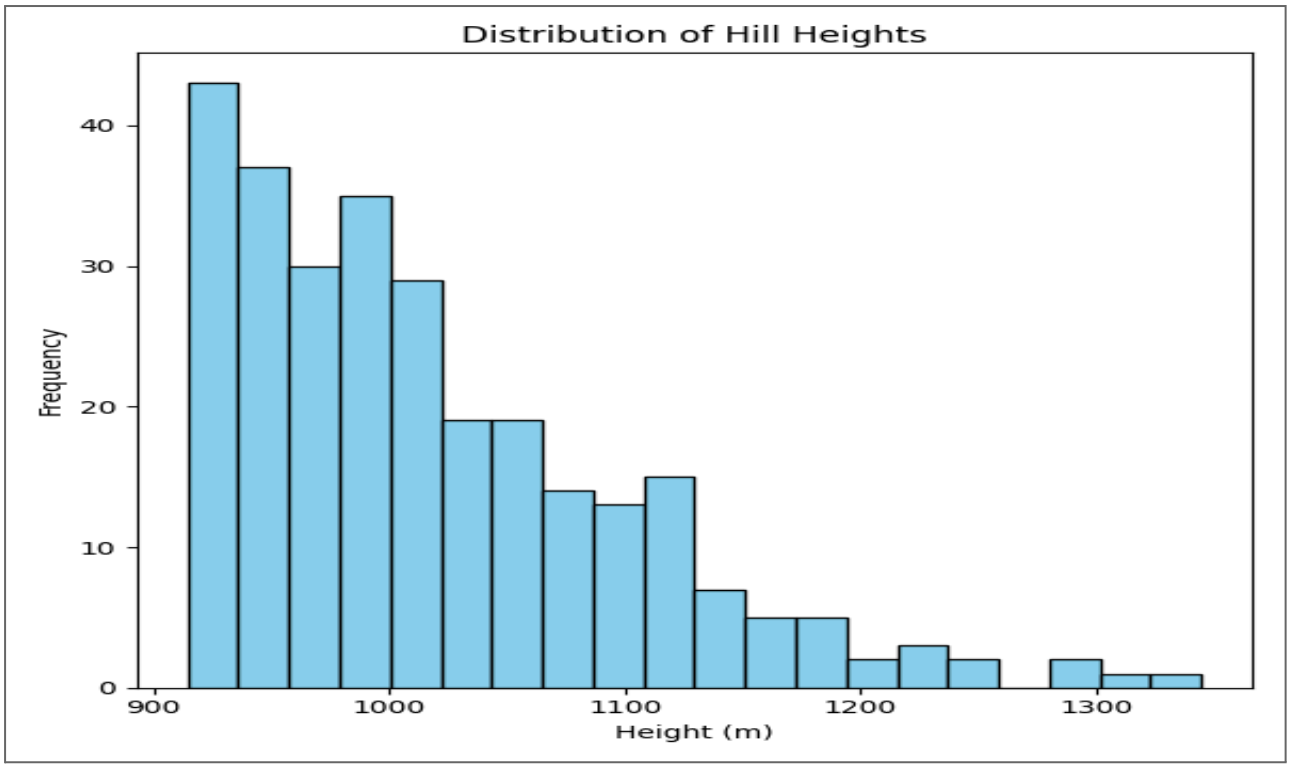

Fig 5.3 : Histogram for Distribution of Hill Heights.

Conclusion:

In conclusion, this scenario provides a deeper understanding of the dataset through advanced analysis techniques. Creating the Height Category feature helps in better classification and comparison of hills. Using NumPy functions like np.diff() allows efficient analysis of height variations. Multiple visualizations such as line graphs, stacked bar charts, and histograms offer different perspectives of the data. These combined methods enhance data interpretation and insight generation. Overall, it demonstrates a comprehensive approach to data analysis.

Final Conclusion:

In conclusion, all scenarios together demonstrate a complete data analysis workflow, starting from data loading and cleaning to advanced visualization. The initial steps ensure the dataset is accurate and reliable for analysis. Basic graphs like line and bar charts help in understanding trends and comparisons. Intermediate visualizations like pie charts show distribution clearly, while advanced techniques provide deeper insights. The use of Pandas, NumPy, and Matplotlib enhances efficiency and analysis capability. Overall, these scenarios build a strong foundation for practical data analysis and visualization skills.

What you have learned

Through this Project, I learned how to load and preprocess datasets using Pandas, including handling missing values and correcting data types. I understood how to use NumPy for efficient numerical operations like array conversion and calculating differences. I gained experience in creating different types of visualizations such as line graphs, bar charts, pie charts, and histograms using Matplotlib. I also learned how to interpret graphs to identify trends, patterns, and distributions. Additionally, I practiced saving visual outputs for reports and presentations. Overall, this improved my practical skills in data analysis and visualization.